TUTORIAL DE GIT

Capítulo 10. Los entresijos internos de Git

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)
Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Este capítulo muestra cómo funciona Git por dentro. En el fondo, Git es un sistema de archivos direccionado por contenido con una interfaz de control de versiones encima. Entenderlo ayuda a comprender por qué los comandos se comportan como lo hacen.

10.1 Fontanería y porcelana

Los comandos de uso diario (add, commit, checkout) se llaman **porcelana**. Debajo existen comandos de bajo nivel, la **fontanería**, pensados para scripts y para entender el funcionamiento interno.

El directorio .git contiene, entre otros:

- **objects/**: la base de datos de contenido.
- **refs/**: punteros a commits (ramas, etiquetas, remotos).
- **HEAD**: indica la rama actual.
- **index**: el área de preparación.
- **config**: la configuración del repositorio.
- **hooks/**: los scripts de enganche.

10.2 Objetos Git

Git es un almacén clave-valor: se le da contenido y devuelve una clave (el SHA-1) con la que se puede recuperar después.

```
echo "contenido de prueba" | git hash-object -w --stdin
git cat-file -p d670460b4b4aece5915caf5c68d12f560a9fe3e4
git cat-file -t d670460b
```

Hay cuatro tipos de objetos:

- **blob:** el contenido de un archivo (sin su nombre).
- **tree:** un directorio; relaciona nombres de archivo con blobs y otros trees.
- **commit:** apunta a un tree raíz, a sus padres e incluye autor, fecha y mensaje.
- **tag:** una etiqueta anotada que apunta a otro objeto.

```
git cat-file -p main^{tree}
git cat-file -p HEAD
```

Cada objeto se comprime con zlib y se guarda en .git/objects, en una carpeta con los dos primeros caracteres del SHA-1 y un archivo con los 38 restantes.

10.3 Referencias Git

Recordar SHA-1 es imposible, por eso existen las referencias: archivos de texto que contienen un SHA-1.

```
cat .git/refs/heads/main
git update-ref refs/heads/main 1a410efb
cat .git/HEAD
git symbolic-ref HEAD
```

- Una **rama** es un archivo en refs/heads con el SHA-1 de su último commit.
- **HEAD** es una referencia simbólica: normalmente contiene "ref: refs/heads/main".
- Las **etiquetas** están en refs/tags y las ramas remotas en refs/remotes.

10.4 Archivos empaquetadores (packfiles)

Para ahorrar espacio, Git agrupa periódicamente los objetos sueltos en un archivo .pack, guardando versiones parecidas como diferencias (deltas) respecto a otras.

```
git gc
git verify-pack -v .git/objects/pack/pack-*.idx
```

10.5 Las especificaciones de referencia (refspec)

Una refspec indica cómo se mapean las ramas del remoto con las locales. En .git/config aparece algo como:

```
fetch = +refs/heads/*:refs/remotes/origin/*
```

El signo + permite actualizar aunque no sea fast-forward. Con refspecs se puede empujar una rama local a una remota con otro nombre o borrar ramas remotas:

```
git push origin main:rama_remota
git push origin :rama_a_borrar
```

10.6 Protocolos de transferencia

Git puede transferir datos con un protocolo **tonto** (descarga archivo por archivo sobre HTTP, solo lectura) o uno **inteligente**, en el que procesos como upload-pack y receive-pack negocian exactamente qué objetos faltan y envían solo esos en un packfile.

10.7 Mantenimiento y recuperación de datos

```
git gc --auto
git count-objects -v
git reflog
git fsck --full
git branch recuperada ab1afef
```

Si se borra una rama o se hace un reset --hard por error, git reflog o git fsck permiten encontrar el SHA-1 del commit perdido y crear una rama nueva que apunte a él.

Para eliminar un archivo grande de toda la historia se reescriben los commits (con git filter-repo) y luego se limpian los objetos inalcanzables.

10.8 Variables de entorno

Variables como GIT_DIR, GIT_WORK_TREE, GIT_AUTHOR_NAME, GIT_EDITOR o GIT_TRACE modifican el comportamiento de Git. Por ejemplo, GIT_TRACE=1 muestra lo que Git hace internamente al ejecutar un comando, muy útil para depurar.

10.9 Resumen

Por dentro, Git guarda blobs, trees, commits y tags identificados por SHA-1; las ramas son simples archivos con un SHA-1; los packfiles comprimen la historia y el reflog permite recuperar casi cualquier cosa. Conocer esta estructura hace que el resto de comandos tenga sentido.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2